

Computational Intelligence

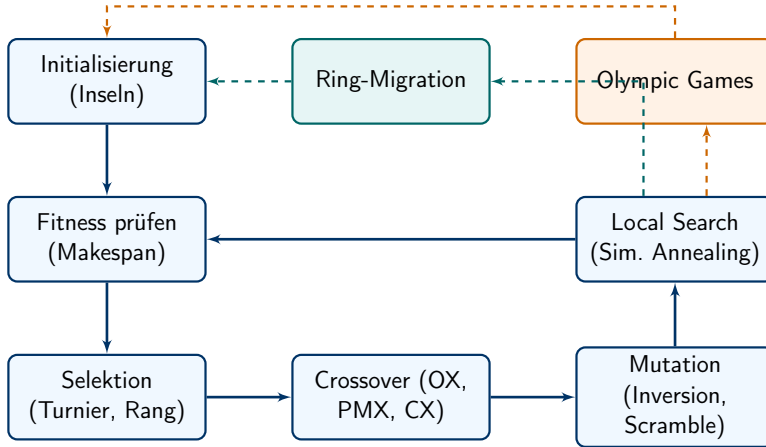
Projekt-Präsentationen

Domenico Avolio, Tim Lukas, Maximilian Murrath

Hochschule für Technik Stuttgart

7. Juni 2026

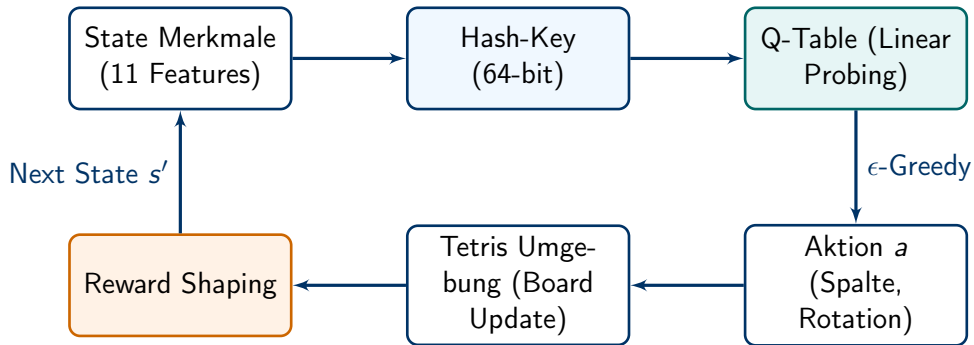
Genetischer Algorithmus: Architektur



Genetischer Algorithmus: Details

- ▶ **Architektur (Archipel-Modell):** Parallele Berechnung mehrerer unabhängiger Insel-Populationen mittels `multiprocessing.Pool`.
- ▶ **Evolutionäre Operatoren:**
 - ▶ *Selektion:* Auswahl durch Turnier-, Rang- oder Roulette-Verfahren.
 - ▶ *Crossover:* Rekombination von Job-Sequenzen (Order Crossover, PMX, Cycle Crossover).
 - ▶ *Mutation & Lokale Suche:* Inversion/Scramble sowie zielgerichtete Optimierung durch Simulated Annealing.
- ▶ **Interaktion der Inseln:**
 - ▶ *Ring-Migration:* Die besten Individuen wandern regelmäßig zur Nachbarinsel.
 - ▶ *Olympic Games:* Periodischer Austausch, bei dem der "Champion" auf alle Inseln kopiert wird.
- ▶ **Dynamische Anpassung:** Erhöhung der Mutationsrate bei Stagnation; kompletter Neustart ("Retirement") von stagnierenden Inseln.

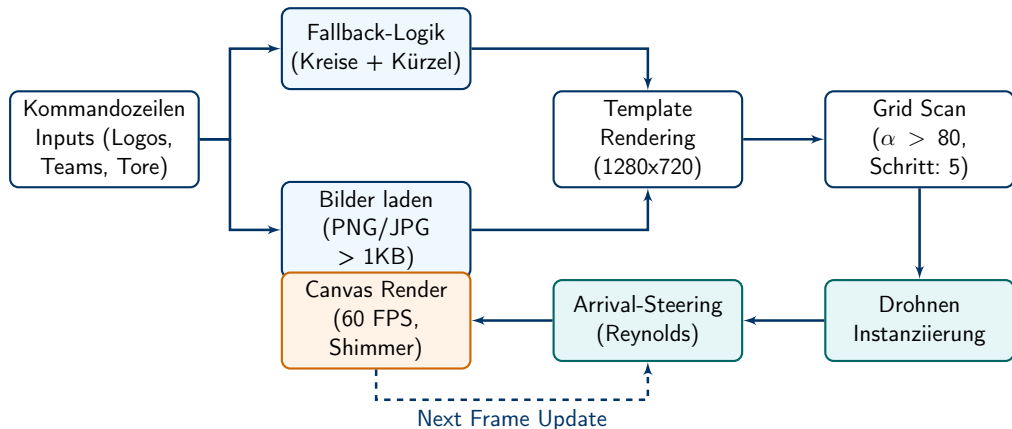
Tetris: Reinforcement Learning



Tetris Reinforcement Learning: Details

- ▶ **Umgebung & Konzept:** Tabulares Q-Learning (implementiert in purem C).
- ▶ **State & Action Space:**
 - ▶ Reduktion des States auf 11 Schlüsselmerkmale (z.B. max. Höhe, Löcher, aktuelle/nächste Steine).
 - ▶ 48 Platzierungs-Entscheidungen pro Schritt (12 Spalten $\times$ 4 Rotationen).
- ▶ **Q-Table & Hash-Mapping:**
 - ▶ Ein 64-bit Hash mappt States über "Linear Probing" auf ein float-Array der Aktionen.
- ▶ **Reward Shaping (Belohnungssystem):**
 - ▶ *Positiv:* Linien-Abbau (+150 bis +1100), Reduktion von Höhe und "Bumpiness".
 - ▶ *Negativ:* Entstehung neuer Löcher, hohe Gesamtbauhöhe, Game Over (-450).
- ▶ **Training:** Bellman-Update mit ϵ -Greedy Exploration und heuristischen Fallbacks für robustes Lernen.

Drohnen-Schwarm-Show: Architektur & Inputs



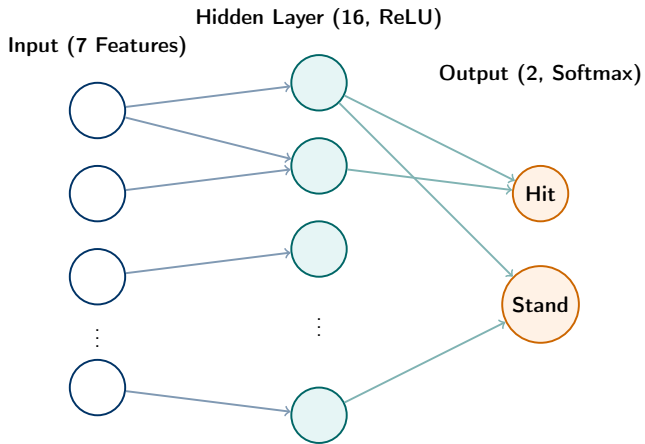
Drohnen-Schwarm-Show: Input & Extraktion

- ▶ **Schritt 1: Kommandozeilen-Inputs parsen**
 - ▶ Parameter: 2 Logo-Dateipfade, 2 Teamnamen, Spielstand, Titel & Untertitel.
 - ▶ *Default-Szenario*: Fehlen Parameter, wird automatisch das historische UEFA-Cup Finale 1989 geladen: **SSC Napoli vs. VfB Stuttgart**.
- ▶ **Schritt 2: Fallback-Logik für fehlende Bilder**
 - ▶ Wird ein Bild nicht gefunden oder ist korrupt (< 1 KB), greift der Fallback.
 - ▶ Erzeugt einen dynamischen Farbkreis mit einem **3-Buchstaben-Kürzel** des Teamnamens (z.B. SSSC"für Napoli).
- ▶ **Schritt 3: Grid Scan & Farbextraktion**
 - ▶ Logos und dynamisch skalierte Schriften werden intern auf ein 1280×720 Template gerendert.
 - ▶ Abtastung in 5×5 Pixel großen Schritten (`SAMPLE_STEP = 5`).
 - ▶ Eine Drohne wird nur generiert, wenn die Deckkraft **Alpha** > 80 ist.
 - ▶ *Korrektur*: Sehr dunkle Pixel ($R, G, B < 30$) werden leicht bläulich aufgehellt (50, 50, 70), um am schwarzen Nachthimmel sichtbar zu bleiben.

Drohnen-Schwarm-Show: Vektormathematik

- ▶ **Startbedingungen:** Zufälliges Delay (bis zu 200 Frames) & Streuung am unteren Bildschirmrand für einen organischen Launch.
- ▶ **Arrival-Steering Behavior (nach Craig Reynolds):**
 - ▶ *Distanzvektor:* $\vec{d} = \vec{x}_{\text{target}} - \vec{x}_{\text{current}}$ mit absoluter Distanz $D = |\vec{d}|$.
 - ▶ *Geschwindigkeit drosseln:* Falls $D < \text{arrivalRadius}$ (100px), wird proportional abgebremst: $v_{\text{desired}} = v_{\text{max}} \cdot (D / \text{arrivalRadius})$. Ansonsten $v_{\text{desired}} = v_{\text{max}}$.
 - ▶ *Lenkkraft (Steer Force):* $\vec{f} = \left(\frac{\vec{d}}{D}\right) v_{\text{desired}} - \vec{v}_{\text{current}}$. Die Kraft wird auf maxForce (0.12) limitiert.
 - ▶ *Update:* Geschwindigkeit wird aktualisiert ($\vec{v} = \vec{v} + \vec{f}$) und durch das Limit $v_{\text{max}} = 3.5$ gedeckelt.
- ▶ **Visualisierung & Shimmer-Effekt (60 FPS):**
 - ▶ Farben interpolieren sich sanft von "Warmweiß" beim Start zur Zielfarbe.
 - ▶ Nach Ankunft am Ziel aktiviert sich ein **Sinus-Flackern** pro Drohne:
 $RGB_{\text{out}} = RGB_{\text{target}} \cdot (0.88 + 0.12 \cdot \sin(\text{frame} \cdot 0.04 + \text{phase}))$.

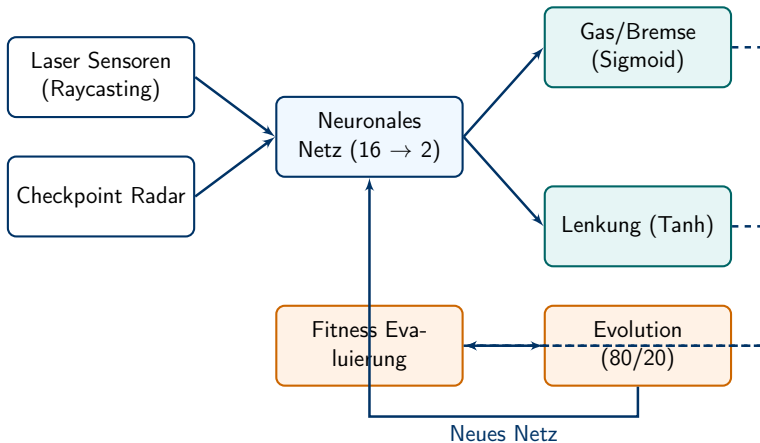
Blackjack: Neuronales Netz



Blackjack Neuronales Netz: Details

- ▶ **Architektur:** Ein in C geschriebenes Feedforward Neuronales Netz ($7 \rightarrow 16 \rightarrow 2$).
- ▶ **Inputs (7 normierte Features):**
 - ▶ Player Total, Usable Ace, Dealer Upcard, Busted Flag, Terminal Flag, Running Count, True Count.
- ▶ **Aufbau der Schichten:**
 - ▶ *Hidden Layer:* 16 Neuronen, aktiviert durch ReLU ($\max(0, x)$).
 - ▶ *Output Layer:* 2 Logits für "Hit und SStand", umgewandelt in Wahrscheinlichkeiten durch eine Softmax-Funktion.
- ▶ **Training (Supervised Learning):**
 - ▶ Berechnung des Fehlers über Cross-Entropy Loss ($L = -\log P(\text{Target})$).
 - ▶ Backpropagation der Gradienten und Aktualisierung der Gewichte durch Stochastic Gradient Descent (SGD).

F1 AI Racing: Neuroevolution



F1 AI Racing: Details

- ▶ **Das Konzept:** Kombination von Neuronalen Netzen mit Genetischen Algorithmen (ähnlich NEAT). Fahrzeuge erlernen die Steuerung komplett autark über Generationen hinweg.
- ▶ **Sensorik (16 Inputs):**
 - ▶ 7 Laser-Sensoren messen Entfernungen zu Streckenbegrenzungen (Raycasting).
 - ▶ Mathematisches Radar peilt den nächsten Checkpoint an (Distanz & Winkel per $\arctan2$).
- ▶ **Fahrphysik:**
 - ▶ Berücksichtigung von Reibung und Grip-Verlust bei hoher Geschwindigkeit (Untersteuern).
 - ▶ *Windschatten-Effekt (Slipstream & Dirty Air):* Extrapush im Sog des Vordermanns, abgeschwächt in Kurven.
- ▶ **Fitness & Selektion:**
 - ▶ Die besten 20% einer Generation überleben ("Survival of the Fittest").
 - ▶ Komplexe Fitnessbewertung (Punkte für Checkpoints, Überholen & Überlebenszeit; massiver Abzug bei Crashes).